
When and Why Does Hindsight Experience Replay Improve Soft Actor-Critic under Sparse Rewards?

Mustafa Asfari

Mohammad Alkhozami

github.com/mustafa99705/-adrl-sac-her-fetchreach

Abstract

We study when and why Hindsight Experience Replay (HER) helps Soft Actor-Critic (SAC) under sparse rewards. We compare four conditions (dense, dense+HER, sparse, sparse+HER) on the simple `FetchReach-v3` task, and focus on the two central sparse-reward conditions on the harder `FetchPush-v3` task. On the easy task, all four conditions solve it near-perfectly; under sparse rewards, HER mainly shortens time-to-competence, reducing the median steps to 90% success from 22,000 to 8,000 ($2.75\times$ faster). On the harder task, plain sparse-reward SAC fails to learn within the 10^6 -step training budget (5.4% mean final success, 0/10 seeds reaching 90%), whereas sparse SAC+HER reaches 90.7% mean final success (99.0% median, 9/10 seeds reaching 90%). This shows a much stronger benefit of HER on the harder task. Tracking SAC's auto-tuned entropy coefficient offers a plausible explanation: without HER it collapses toward zero while the policy is still failing, whereas with HER it remains substantially higher throughout training. A relabeling-ratio ablation ($n_{\text{sampled_goal}} \in \{1, 4, 8\}$) shows that the benefit of additional relabeling saturates rather than growing indefinitely.

1 Introduction

In real robotic tasks, the most natural reward specification is often binary: the task was either accomplished or it was not. Dense reward functions can provide more feedback, but they usually have to be hand-engineered, require domain knowledge, and may bias the learned behavior in unintended ways. However, standard off-policy actor-critic methods such as Soft Actor-Critic (SAC) can struggle under sparse rewards because the replay buffer may contain many weakly informative failed transitions before the agent observes its first success. While Hindsight Experience Replay (HER) has been shown to improve learning in sparse-reward settings, it is less clear under which task conditions its benefit is most significant and what training dynamics accompany that benefit. This report investigates not only *whether* HER improves SAC, but also *when* its contribution becomes meaningful as task difficulty increases and *why* it helps, by analyzing the training dynamics associated with HER.

2 Related Work

This study connects three main topics from the lecture and the RL literature: off-policy maximum-entropy reinforcement learning, goal-conditioned reinforcement learning, and reward design under sparse feedback.

Soft Actor-Critic (SAC) (1), an off-policy, maximum-entropy actor-critic algorithm, serves as the learning backbone for all experiments in this study.

Hindsight Experience Replay (HER) (2) addresses reward design under sparse feedback: it provides a goal-relabeling mechanism that allows failed trajectories to be reused by replacing the original desired goal with goals that were actually achieved, turning sparse-reward failures into useful training signal.

This study also relates to goal-conditioned reinforcement learning and to Universal Value Function Approximators (4), since both the policy and the action-value function must condition on the desired goal, enabling the same experience to be evaluated with respect to different goals.

The Fetch environments and their sparse and dense reward variants are based on the multi-goal robotics benchmark introduced by Plappert et al. (3) and are available through Gymnasium-Robotics.

3 Contribution and Attribution

We build on existing, unmodified implementations rather than reimplementing published algorithms. SAC and the `HerReplayBuffer` are provided by Stable-Baselines3 (6), while the Fetch environments and their sparse/dense reward variants are provided by Gymnasium-Robotics, following the multi-goal benchmark of Plappert et al. (3). The hyperparameters are adopted from the tuned Fetch settings of RL Baselines3 Zoo (5); we did not perform a separate hyperparameter search. Reimplementing SAC or HER from scratch was explicitly outside the scope of this study. Our contributions are the experimental design and the infrastructure required to run, diagnose, and interpret the experiments: (1) identifying and fixing a reset bug in `gymnasium-robotics` 1.3.x that silently made both tasks harder than documented (`env_fix.py`, Section 5); (2) developing a unified training and evaluation pipeline (`train.py`) supporting both tasks, all experimental conditions, and the relabeling-ratio ablation under one shared configuration; (3) implementing `HerDiagnosticsCallback`, which logs the entropy-coefficient and goal-distance-percentile trajectories used for the training-dynamics analysis in Section 6.2; this diagnostic is not part of Stable-Baselines3 and was developed specifically for this study; (4) designing and executing all 70 main training runs, together with the separate two-run pilot used to calibrate the FetchPush training budget, and developing the analysis and plotting pipeline (`plot_results.py`); and (5) interpreting the results in Section 6, including the association between HER and the observed entropy dynamics, as well as the finding that the benefit of increasing the relabeling ratio saturates.

4 Approach

We compare four conditions—dense SAC, dense SAC+HER, sparse SAC, and sparse SAC+HER—on `FetchReach-v3`, a relatively simple reaching task. On `FetchPush-v3`, where the robot must push an object to a target location and sparse-reward learning is known to be substantially harder (2), we focus on the two central sparse-reward conditions, sparse SAC and sparse SAC+HER.

Algorithmic hyperparameters are identical across task, reward type, and HER on/off, so that the reward signal and relabeling strategy remain the main experimental variables within each comparison. We tested three hypotheses stated in the project proposal: (i) on the simple reaching task, HER provides only a modest speedup, since random exploration occasionally reaches the goal on its own; (ii) on the harder pushing task, HER is expected to substantially improve learning performance; and (iii) HER’s effect is visible in the training dynamics, e.g. in the evolution of SAC’s entropy coefficient and in the distribution of relabeled goals, potentially reflecting an implicit curriculum.

Section 6 strongly supports hypotheses (i) and (ii). For hypothesis (ii), the observed effect was considerably larger than initially expected: HER made the difference between successful learning and near-total failure within the training budget. The results provide partial support for hypothesis (iii), with the entropy-coefficient dynamics showing the clearest signal and the implicit-curriculum evidence being weaker.

Algorithm 1 SAC with optional Hindsight Experience Replay

Require: environment e , SAC algorithm A , reward function r , HER flag h , relabels per transition k
Ensure: trained goal-conditioned policy $\pi(a \mid s, g)$
Initialize replay buffer R , SAC actor and critic networks
while training budget not reached **do**
 Sample goal g and initial state s_0 from e
 Roll out one episode $(s_0, a_0, \dots, s_T)$ with $\pi(\cdot \mid s, g)$
 Store $(s_t, a_t, r(s_t, a_t, g), s_{t+1}, g)$ in R for all t
 if h is true **then**
 for each t and each of k relabels **do**
 sample $\tau \sim \mathcal{U}(t+1, T)$; $g' \leftarrow$ achieved goal in s_τ
 store $(s_t, a_t, r(s_t, a_t, g'), s_{t+1}, g')$ in R $\triangleright$ often a success: $r = 0$
 end for
 end if
 Update SAC actor and critics on minibatches from R
end while
return π

Algorithm 1 illustrates HER conceptually. In our Stable-Baselines3 implementation, real environment transitions are stored in the replay buffer, while relabeled virtual transitions are generated when training minibatches are sampled from `HerReplayBuffer`. Thus, the conceptual relabeling shown above is performed during replay sampling rather than by explicitly storing additional relabeled transitions at rollout time.

5 Experimental Setup

Both tasks use Stable-Baselines3 SAC (6) with a `MultiInputPolicy` for the dictionary observations and share one fixed hyperparameter configuration adopted from the tuned RL Baselines3 Zoo settings for Fetch-style tasks (5). We use $\gamma = 0.95$, a learning rate of 10^{-3} , batch size 256, and replay buffer size 10^6 . For HER runs, we use $n_{\text{sampled_goal}} = 4$ with the *future* goal-selection strategy unless otherwise stated; all remaining settings follow the Stable-Baselines3 defaults (6). We deliberately did not tune hyperparameters separately for individual conditions, so that within each task comparison the reward setting and the use of HER remain the main experimental variables. The success criterion is the one defined by each environment: the end effector in FetchReach, or the manipulated object in FetchPush, must be within 5 cm of the desired goal.

We additionally found and fixed a reset bug in `gymnasium-robotics 1.3.x` (the only release line exposing the `-v3` Fetch environments): the initial arm pose was not restored on reset, so every episode started with the gripper approximately 0.55 m from the goal-sampling center instead of the documented ≤ 0.26 m — a roughly $2.1\times$ larger initial offset (0.55/0.26). The fix (`env_fix.py`) backports the official upstream correction and is applied identically to every run reported below.

Experimental Scope. The original proposal specified 5 seeds per condition and an ablation over $n_{\text{sampled_goal}} \in \{1, 4\}$. We extended the final study to 10 seeds per main condition and added $n_{\text{sampled_goal}} = 8$ to the ablation in order to obtain a more robust estimate of the substantial seed-to-seed variability observed on FetchPush.

On `FetchReach-v3`, we ran all four conditions—dense SAC, dense SAC+HER, sparse SAC, and sparse SAC+HER—with 10 seeds each. Each run used a training budget of 100,000 environment steps, with deterministic evaluation on 20 held-out episodes every 2,000 steps.

On `FetchPush-v3`, we focused on the two central sparse-reward conditions, sparse SAC and sparse SAC+HER, again using 10 seeds each. A separate two-run pilot, one seed per condition and 200,000 steps per run, showed that success remained close to zero at that budget (0% for sparse SAC and 5% for sparse SAC+HER), while training throughput was approximately 54–61 environment steps per second. Based on this pilot, we used a main training budget of 10^6 environment steps per run. Evaluation was performed every 10,000 steps rather than every 2,000 steps, keeping evaluation overhead proportionate to the approximately ten-times-longer training budget.

For the HER ablation on FetchPush-v3, we evaluated $n_{\text{sampled_goal}} \in \{1, 4, 8\}$ using 5 seeds per setting. The $n_{\text{sampled_goal}} = 4$ condition reuses the first five seeds of the main sparse SAC+HER experiments rather than repeating those runs. The final main study therefore consists of 70 training runs, in addition to the separate two-run pilot.

To support the training-dynamics analysis, the runs additionally logged SAC’s automatically tuned entropy coefficient α and statistics of $\|\text{achieved goal} - \text{desired goal}\|$ in replay-buffer minibatches. These diagnostics were recorded every 2,000 steps on FetchReach and every 10,000 steps on FetchPush.

6 Results

6.1 Main comparison: does HER’s benefit grow with task difficulty?

Metric definitions used in all tables below: per-seed *success* is the mean evaluation success rate over the last 5 evaluation points of a run; *Mean/Median success* aggregate this across seeds within a condition (we report both because Push has occasional catastrophic-failure seeds that pull the mean down without affecting the median – see Section 7). *Median steps to 90%* is computed only over seeds whose evaluation curve reaches 90% success at some point; the count of seeds that do so is reported separately in the *Seeds $\geq 90\%$* column, and seeds that never reach it are excluded from the median (not treated as ∞). *AUC* is the evaluation success-rate curve integrated (trapezoidal rule) over training steps and normalized by the total training budget, so $\text{AUC} \in [0, 1]$.

Table 1: As specified in the proposal: 5 seeds/condition.

Task	Condition	Mean success	Median success	Seeds $\geq 90\%$	Steps to 90%	AUC
Reach	dense	100.0%	100.0%	5/5	6,000	0.944
Reach	dense + HER	100.0%	100.0%	5/5	6,000	0.945
Reach	sparse	98.8%	100.0%	5/5	22,000	0.804
Reach	sparse + HER	100.0%	100.0%	5/5	8,000	0.931
Push	sparse	6.2%	6.0%	0/5	— (never)	0.065
Push	sparse + HER	82.8%	99.0%	4/5	380,000	0.553

At the protocol’s originally specified scale, the central finding is already clear: on FetchReach, HER mainly accelerates sparse-reward learning, whereas on FetchPush it changes the outcome from near-failure to successful learning within the available training budget. Tables 2 and 3 extend the main comparisons to 10 seeds per condition to obtain a more robust estimate of seed-to-seed variability, particularly on the noisier FetchPush task. The overall conclusion remains unchanged.

Table 2: Extended robustness check: same runs, 10 seeds/condition (FetchReach-v3). All conditions solve the task; HER mainly affects how fast.

Condition	Mean success	Median success	Seeds $\geq 90\%$	Steps to 90%	AUC
dense	100.0%	100.0%	10/10	6,000	0.947
dense + HER	100.0%	100.0%	10/10	6,000	0.945
sparse	99.4%	100.0%	10/10	22,000	0.826
sparse + HER	100.0%	100.0%	10/10	8,000	0.929

Table 3: Extended robustness check: same runs, 10 seeds/condition (FetchPush-v3). Sparse SAC essentially fails to learn; HER recovers most of the task.

Condition	Mean success	Median success	Seeds $\geq 90\%$	Steps to 90%	AUC
sparse	5.4%	6.0%	0/10	— (never)	0.067
sparse + HER	90.7%	99.0%	9/10	350,000	0.610

Table 2 confirms hypothesis (i): on the easy task, every condition eventually succeeds, and HER’s main contribution under sparse rewards is a roughly $2.75\times$ reduction in steps-to-90% success (22,000 $\rightarrow$ 8,000), rather than a change in final task solvability. Table 3 and Figure 1b strongly support

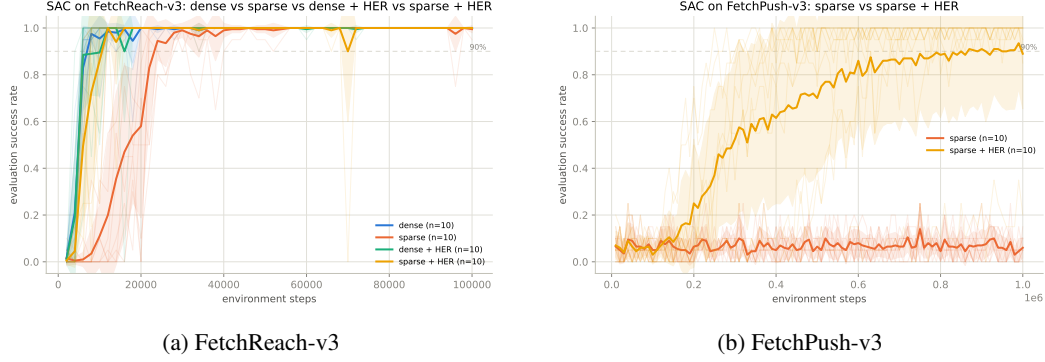


Figure 1: Evaluation success rate vs. environment steps (thin lines: individual seeds; thick line: mean; band: mean $\pm$ 1 std. dev. across seeds). On Reach, all HER and dense conditions overlap and converge fast, with plain sparse trailing but still reaching near-perfect success. On Push, sparse SAC (orange) never rises above noise around 5–10%, while sparse+HER (yellow) climbs steadily to $\sim$ 90%.

hypothesis (ii): without HER, SAC extracts very little useful learning signal from the sparse pushing reward within the 10^6 -step training budget, reaching only 5.4% mean final success, close to the pre-training baseline. In contrast, HER on the identical sparse reward signal reaches 90.7% mean final success (99.0% median). The gap between mean and median is caused by a catastrophic-failure seed, discussed in Section 7. Thus, HER’s benefit on FetchPush is not merely a speedup; within the training budget, it separates near-failure from successful learning.

6.2 Mechanism: why does HER help?

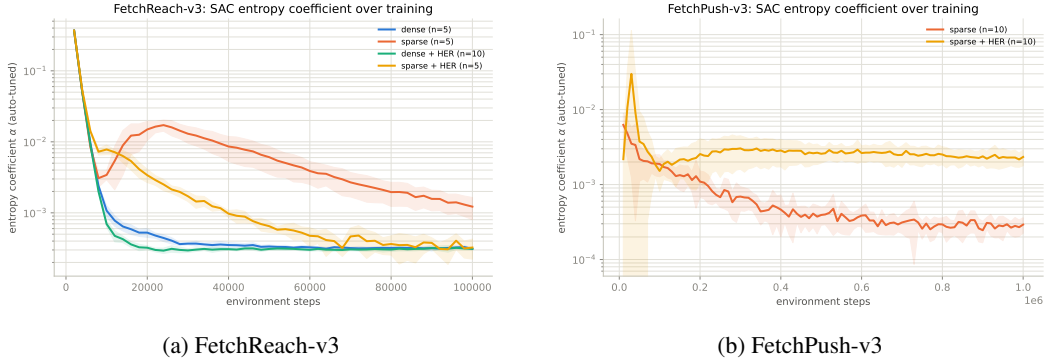


Figure 2: SAC’s automatically tuned entropy coefficient α over training (log scale; mean $\pm$ one standard deviation across seeds). On FetchReach, α decreases strongly across conditions while the task is successfully learned. On FetchPush, sparse SAC without HER converges to a substantially lower entropy coefficient than sparse SAC+HER while remaining unsuccessful.

Figure 2 is consistent with a plausible mechanistic account of hypothesis (ii). On FetchPush, plain sparse-reward SAC receives very few informative successful transitions and remains near failure throughout training, reaching only 5.4% mean final success. At the same time, SAC’s automatically tuned entropy coefficient α decreases to a very small value, indicating that the policy becomes increasingly low-entropy despite not having solved the task. This behavior is consistent with a premature-convergence failure mode.

With HER, relabeled transitions provide substantially more informative training samples, and α remains markedly higher over much of training while the policy improves toward high success. This pattern is consistent with HER being associated with healthier entropy dynamics and a more stochastic policy during successful learning.

Importantly, entropy reduction itself is not necessarily problematic. On FetchReach (Figure 2a), α also decreases strongly across conditions, but there this reduction coincides with successful task learning. In other words, entropy reduction tracks genuine competence on Reach, whereas on FetchPush without HER it occurs while the policy is still failing. We therefore interpret Figure 2 as evidence of an association between HER and healthier entropy dynamics, not as a controlled test establishing entropy as the causal mechanism behind HER’s performance benefit.

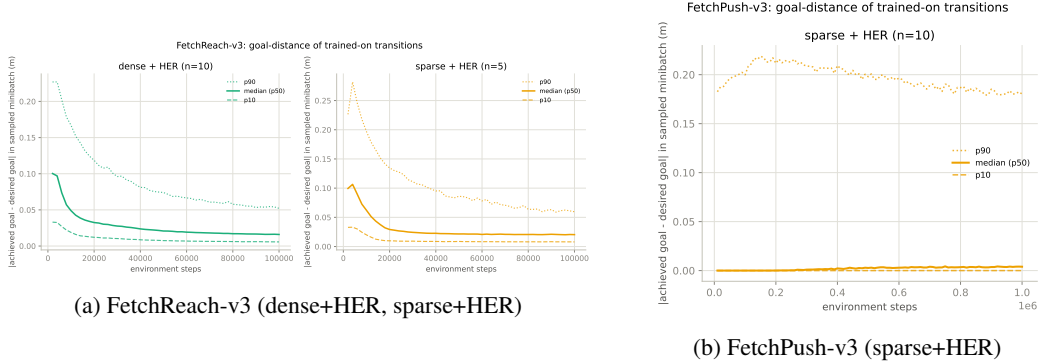


Figure 3: The p10, median (p50), and p90 of $\|\text{achieved goal} - \text{desired goal}\|$ across transitions sampled for training. The three percentiles are shown as separate lines rather than as a shaded band so that changes in the upper tail remain visible when the median is close to zero.

For the implicit-curriculum part of hypothesis (iii), the pattern differs by task. On FetchReach (Figure 3a), the median trained-on goal distance decreases clearly as training progresses, from approximately 0.10 m to approximately 0.016–0.020 m for dense+HER and sparse+HER, respectively. This pattern is consistent with an implicit curriculum: as the policy improves, the transitions it is trained on become associated with progressively smaller goal errors.

On FetchPush (Figure 3b), the same effect is much less visible in the pooled distribution. With $n_{\text{sampled_goal}} = 4$, approximately 80% of sampled training transitions are virtual HER samples. In our diagnostics, many of these samples have goal distances close to zero, which keeps both the median and p10 near zero for most of training. The upper tail of the distribution, captured by p90, is therefore more informative: it increases from roughly 0.18 m to a peak of approximately 0.21–0.215 m around 150–250k steps and then decreases toward approximately 0.18 m by one million steps.

This weaker pattern suggests changing goal-distance statistics during training, but the pooled metric does not isolate the non-relabeled transitions and therefore cannot provide strong evidence that an implicit curriculum is the mechanism responsible for HER’s performance gain on FetchPush. We treat this as a limitation of the diagnostic rather than as evidence against hypothesis (iii). On the harder task, the entropy-coefficient signal in Figure 2 provides the clearer training-dynamics indicator.

6.3 Ablation: does more relabeling always help?

The proposal specified an ablation over $n_{\text{sampled_goal}} \in \{1, 4\}$ with 5 seeds per setting. We additionally evaluated $n_{\text{sampled_goal}} = 8$ using the same number of seeds to test whether the effect of additional relabeling continues beyond the originally specified range.

Table 4: HER relabeling-ratio ablation, FetchPush-v3 sparse+HER, 5 seeds/setting (rows 1–4 are the specified range; row 8 is the extension).

$n_{\text{sampled_goal}}$	Mean success	Median success	Seeds $\geq 90\%$	Steps to 90%	AUC
1	84.0%	99.0%	4/5	400,000	0.566
4	82.8%	99.0%	4/5	380,000	0.553
8	88.6%	99.0%	4/5	260,000	0.637

Table 4 and Figure 4 show that increasing the relabeling ratio does not produce a clear monotonic or dramatic improvement in final performance. The setting $n_{\text{sampled_goal}} = 8$ learns somewhat faster,

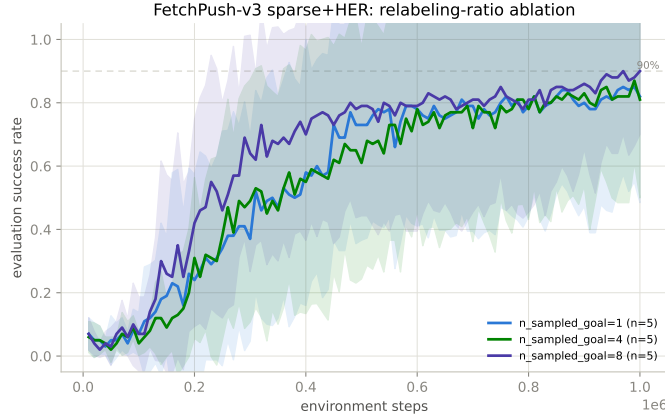


Figure 4: Success rate for the HER relabeling-ratio ablation on FetchPush-v3 (mean $\pm$ one standard deviation across 5 seeds). The wide variability bands reflect substantial seed-to-seed variation. Although $n_{\text{sampled_goal}} = 8$ reaches high success faster, all three settings achieve the same median final success.

reaching 90% success after a median of 260,000 environment steps, compared with approximately 380,000–400,000 steps for the smaller ratios. It also achieves a somewhat higher AUC and mean final success. However, $n_{\text{sampled_goal}} = 1$ and 4 show no clear separation with the available five seeds, and all three settings converge to a similar final success region by one million environment steps. In particular, the median final success is 99.0% for all three relabeling ratios, and 4 of 5 seeds in each setting reach at least 90% success. Each setting contains one low-performing seed that fails to reach 90% success, with final success between approximately 0.20 and 0.45 for that seed, compared with at least 0.96 for the other four seeds in the same group. This is reflected in the wide variability bands in Figure 4 and indicates that seed-to-seed optimization variability on FetchPush is substantial relative to the differences caused by the relabeling ratio. Overall, the ablation suggests diminishing returns from additional hindsight relabeling. Increasing $n_{\text{sampled_goal}}$ can improve sample efficiency, as seen most clearly for $n_{\text{sampled_goal}} = 8$, but it does not consistently improve final task performance within the tested range.

7 Discussion and Limitations

The central result—HER’s benefit growing from a modest speedup on FetchReach to the difference between near-failure and successful learning on FetchPush within the training budget—strongly supports hypotheses (i) and (ii) from the proposal and provides partial support for hypothesis (iii). The entropy-coefficient trajectories in Section 6.2 offer a plausible mechanistic account of this difference: HER is associated with avoiding the premature, overconfident entropy collapse observed for plain sparse-reward SAC when successful sparse-reward transitions remain very rare.

We note three main limitations. First, the FetchPush experiments show occasional catastrophic-failure seeds, with final success as low as 0.20 even with HER. With 10 seeds in the main comparison and 5 seeds in the ablation, such outliers noticeably reduce the mean while leaving the median largely unchanged. Tables 3 and 4 therefore report both mean and median performance.

Second, the goal-distance pattern is consistent with an implicit curriculum on FetchReach, but is much less informative on FetchPush because the pooled distribution is dominated by HER-reabeled samples. A cleaner mechanism analysis on harder tasks would track non-reabeled (“real-goal”) transitions separately rather than combining them with relabeled transitions.

Third, we deviated from the proposal’s originally specified evaluation cadence on FetchPush. Instead of evaluating every 2,000 steps, we evaluated every 10,000 steps in order to keep evaluation overhead proportionate to the approximately ten-times-longer training budget. This does not change the underlying training procedure or final success measurements, but it reduces the temporal resolution

of metrics such as steps-to-90% success and therefore represents a deviation from the originally specified protocol.

8 Reproducibility

All code, per-run logs, metrics, configuration files, and the LaTeX source of this report are included in the project repository (see the link on the title page). Trained model checkpoints are excluded from the repository because of their size (approximately 540 MB across the 70 main runs), but are available on request.

The README.md documents the software stack, including the versions required for the gymnasium-robotics 1.3.x -v3 environments, together with the exact commands needed to reproduce all experiments. EXPERIMENT_LOG.md records the chronological history of experimental decisions, calibration results, and cluster job IDs underlying the results reported here. The script plot_results.py regenerates all figures and summary tables directly from the stored per-run data.

All 70 main training runs, together with the separate two-run pilot, were executed on the GWDG/KISSKI HPC cluster. Each run used one NVIDIA A100 GPU, 4 CPU cores, and 16 GB RAM. The total computational cost was approximately 115 GPU-hours.

9 Conclusion

This report set out to answer not only *whether* HER helps SAC under sparse rewards, but also *when* its contribution becomes substantial and *why* it helps. Across the 70 main runs, the results show that HER’s benefit increases markedly from FetchReach to FetchPush. On the easy reaching task, HER mainly improves sample efficiency under sparse rewards, reducing the median steps to 90% success from 22,000 to 8,000, while all conditions eventually achieve near-perfect performance. On the harder pushing task, the effect is much larger: within the one-million-step training budget, sparse SAC remains near failure, whereas sparse SAC+HER achieves high success in 9 of 10 seeds.

The mechanism analysis provides partial evidence for why this difference emerges. On FetchPush without HER, SAC’s automatically tuned entropy coefficient decreases to a very small value while the policy is still failing. With HER, the entropy coefficient remains substantially higher over much of training while successful learning progresses. This pattern is consistent with HER being associated with healthier entropy dynamics and a more stochastic policy during learning, but it does not by itself establish entropy or exploration as the causal mechanism behind HER’s performance benefit.

The relabeling-ratio ablation further shows that increasing $n_{\text{sampled_goal}}$ does not lead to a clear monotonic improvement in final performance. The setting $n_{\text{sampled_goal}} = 8$ learns faster and achieves the highest AUC in our experiments, but all tested values $n_{\text{sampled_goal}} \in \{1, 4, 8\}$ reach the same 99% median final success. This suggests diminishing returns from additional hindsight relabeling rather than performance scaling proportionally with the relabeling ratio.

Natural next steps are: (i) to repeat the mechanism analysis while tracking non-relabeled transitions separately, which would provide a cleaner goal-distance signal on FetchPush; (ii) to test whether the stronger HER benefit observed from FetchReach to FetchPush extends to an even harder goal-conditioned task such as FetchSlide-v3 or FetchPickAndPlace-v3; and (iii) to quantify uncertainty more formally using bootstrap confidence intervals or other seed-level statistical analyses, especially given the catastrophic-failure-seed variability observed on FetchPush.

References

- [1] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *ICML*, 2018.
- [2] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, W. Zaremba. Hindsight Experience Replay. *NeurIPS*, 2017.
- [3] M. Plappert, M. Andrychowicz, A. Ray, B. McGrew, B. Baker, G. Powell, J. Schneider, J. Tobin, M. Chociej, P. Welinder, V. Kumar, W. Zaremba. Multi-Goal Reinforcement Learning: Challenging Robotics Environments and Request for Research. *arXiv:1802.09464*, 2018.

- [4] T. Schaul, D. Horgan, K. Gregor, D. Silver. Universal Value Function Approximators. *ICML*, 2015.
- [5] A. Raffin. RL Baselines3 Zoo: A Training Framework for Stable-Baselines3 Reinforcement Learning Agents. <https://github.com/DLR-RM/rl-baselines3-zoo>, 2020.
- [6] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR*, 22(268), 2021.

Checklist

This checklist is not mandatory, but can help you set up your project in a reproducible manner.

1. General points:
 - (a) Do the main claims made in the abstract and introduction accurately reflect your contributions and scope? [\[Yes\]](#)
 - (b) Did you cite all relevant related work? [\[Yes\]](#)
 - (c) Did you describe the limitations of your work? [\[Yes\]](#) [Section 7](#)
 - (d) Did you include a discussion of future work? [\[Yes\]](#) [Section 9 \(Conclusion\)](#)
2. If you are including theoretical results...
 - (a) Did you state the full set of assumptions of all theoretical results? [\[NA\]](#) [purely empirical study; no theoretical claims]
 - (b) Did you include complete proofs of all theoretical results? [\[NA\]](#) [purely empirical study; no theoretical results]
3. If you ran experiments (e.g. for benchmarks)...
 - (a) Did you include the code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL)? [\[Yes\]](#) [GitHub repository linked on the title page; README.md and EXPERIMENT_LOG.md document the exact commands and experimental decisions]
 - (b) Did you specify all the training details (e.g., data splits, hyperparameters, how they were chosen)? [\[Yes\]](#) [Section 5; hyperparameters were adopted from RL Baselines3 Zoo and no separate hyperparameter search was performed, as described in Section 3]
 - (c) Did you run at least 10 repetitions of your method? [\[Yes\]](#) [the main FetchReach and FetchPush comparisons use 10 seeds per condition; the separate relabeling-ratio ablation uses 5 seeds per setting, as specified in the experimental design]
 - (d) Did you report error bars (e.g., with respect to the random seed after running experiments multiple times)? [\[Yes\]](#) [training curves report variability across random seeds; Figures 2 and 4 use mean $\pm$ one standard deviation, while Figure 3 reports the p10, median, and p90 trajectories. Seed-to-seed variability is discussed explicitly in Section 7]
 - (e) Did you include the total amount of compute and the type of resources used (e.g., type of GPUs, internal cluster, or cloud provider)? [\[Yes\]](#) [Section 8: approximately 115 GPU-hours; one NVIDIA A100 GPU, 4 CPU cores, and 16 GB RAM per run on the GWDG/KISSKI HPC cluster]
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets...
 - (a) If your work uses existing assets, did you cite the creators? [\[Yes\]](#) [Section 3: Stable-Baselines3, Gymnasium-Robotics, and RL Baselines3 Zoo]
 - (b) Did you make sure the license of the assets permits usage? [\[Yes\]](#) [Stable-Baselines3 and Gymnasium-Robotics are both MIT-licensed, which permits their use in this academic project]
 - (c) Did you reference the assets directly within your code and repository? [\[Yes\]](#) [exact software versions are pinned in `requirements.txt`, and the corresponding packages are imported directly in `train.py`]